



- Tutorial 10 - Metaheuristics -

Numerical Methods and Optimization
May 12, 2024 — ENSIA

- Metaheuristics -

Exercise 1: General Questions

(1)- Exact methods of problem resolution: Branch and Bound, Branch and Cut, Dynamic programming, Linear Programming, Nonlinear Programming, Lagrangian relaxation...etc

(2)- The goal of optimization methods is to introduce the principle of solving a given problem by using a search space. This search space allows for identifying the states of a given system and finding one or more solutions among these states.

(3)- Heuristics strongly depend on the specific problem. Metaheuristics are problem-independent, making them versatile and applicable to any problem.

(4)- Trajectory methods manipulate one solution at a time and iteratively attempt to improve upon it. They construct a trajectory in the solution space, aiming to move towards optimal solutions. Methods that work with a population of solutions at all times have a "base" of several solutions, referred to as a population.

(5)- Evaluation of a metaheuristic: Ability to find good/optimal solutions, Comparison with optimal solutions (if available), Comparison with Lower/Upper bounds if already known, Comparison with other metaheuristics methods, Run time of the algorithm.

(6)-

- **Local search:** Start with an initial configuration, search neighborhood, and select the best neighbor as a candidate. Stop if no improvement can be found or after some fixed time
- **Tabu Search:** Is a search method based on local search, it introduces a tabu list of neighbors to discourage the search from coming back to previously visited solutions.
- **Memoryless:** No information extracted dynamically is used during the search.

(7)- Aspiration criterion allows to check whether a Tabu state is better than the best state, and if so, to select it, by updating the corresponding Tabu list.

(8)- Ant Colony Optimization: is a metaheuristic algorithm that simulates the foraging behavior of an ant colony to discover the shortest path to the food. As ants search for food, they leave behind pheromones that attract other ants to follow their path. Specifically, ants are more likely to select a path with a higher concentration of pheromones. As more ants move in paths with higher pheromone intensities, the pheromone in those pathways increases, which increases the chance that other ants will choose the same path.

(7)- True or False

- False: Tabu search is proposed to avoid the problem of stagnating in a local minimum encountered in SA.
- False: Metaheuristics gives near-optimal solution and there is no proof of optimality.
- False: Greedy algorithms typically (but not always) fail to find the globally optimal solution because they usually do not operate exhaustively on all the data.

Exercise 2: Solving Traveling Salesman Problem with SA, TS and ACO

(1)- Traveling Salesman Problem TSP: Given the coordinates of n cities, find the shortest closed tour which visits each once and only once.

The model:

$$x_{ij} = \begin{cases} 1 & \text{the path goes from city } i \text{ to city } j \\ 0 & \text{otherwise.} \end{cases}$$

$$\begin{aligned}
 \min \quad & \sum_{i=1}^n \sum_{j \neq i, j=1}^n c_{ij} x_{ij}; \\
 & x_{ij} \in \{0, 1\} && i, j = 1, \dots, n; \\
 & \sum_{i=1, i \neq j}^n x_{ij} = 1 && j = 1, \dots, n; \\
 & \sum_{j=1, j \neq i}^n x_{ij} = 1 && i = 1, \dots, n; \\
 & u_i - u_j + 1 \leq (n-1)(1 - x_{ij}) && 2 \leq i \neq j \leq n; \\
 & 2 \leq u_i \leq n && 2 \leq i \leq n.
 \end{aligned}$$

The first set of equalities requires that each city is arrived at from exactly one other city, and the second set of equalities requires that from each city there is a departure to exactly one other city. The last constraint enforces that there is only a single tour covering all cities, and not two or more disjointed tours that only collectively cover all cities.

(2)- The number of all possible solutions for 5 cities is $5! = 120$.

(3)- Propose solution encoding for the TSP: Path representation (PR) is by far the most used encoding for TSP. PR uses a vector of length n that encodes the cities in the order they are visited. For example, a tour passing through five cities, say in order by 1, 4, 3, 5, 2 and finally returns to the first, can be represented by (1 — 4 — 3 — 5 — 2).

(4)- Let $x_0 = 1 - 2 - 3 - 4 - 5 - 1$, find all neighbors of x_0 using sub-tours inversion:

Initial solution initiale : 1-2-3-4-5-1 (34)

Sub-tours inversion:

1-4-3-2-5-1 (31)

1-3-2-4-5-1 (32)

1-2-5-4-3-1 (25)

1-2-4-3-5-1 (36)

1-2-3-5-4-1 (34)

5- Simulated Annealing

1. The probabilities of accepting each neighbor with $T = 20$:

We know that $p = e^{\frac{Z(x_0) - Z(x_{Neighbor})}{T}}$

$$1-4-3-2-5-1 \text{ (31)} \quad p = e^{\frac{34-31}{20}} = e^{0.15} = 1.50$$

$$1-3-2-4-5-1 \text{ (32)} \quad p = e^{-\frac{34-32}{20}} = e^{0.1} = 1.11$$

$$\mathbf{1-2-5-4-3-1 \text{ (25)}} \quad p = e^{\frac{34-25}{20}} = e^{0.45} = 1.56$$

$$1-2-4-3-5-1 \text{ (36)} \quad p = e^{\frac{34-36}{20}} = e^{-0.1} = 0.90$$

$$1-2-3-5-4-1 \text{ (34)} \quad p = e^{\frac{34-34}{20}} = e^0 = 1$$

2. The execution is given in the C++ programs.
3. The results are shown in the following table:

Set of parameters	Best found solution	Exact solution	Average runtime	$e_{rel}\%$
Γ_1	25	25	0.094	0
Γ_2	25	25	0.099	0

Study the results: With both sets of parameters, SA algorithm converges to the optimal solution.

6- Tabu Search

1. The execution is given in the C++ programs.
2. The results are shown in the following table:

Set of parameters	Best found solution	Exact solution	Average runtime	$e_{rel}\%$
Γ_1	25	25	0.2753	0%
Γ_2	26	25	0.2605	4%

Study the results: With the set of parameters Γ_1 , TS algorithm converges to the optimal solution. However, if we change the tabu list size in Γ_2 the optimal solution is not reached.

7- Ant Colony Optimization

1. The equivalent of a solution in an ant colony is a path to the food source.
2. The equivalent of communication between ants in ACO is the parameter τ_{ij} the existing pheromone trail between cities i and j
3. The number of ants needed is at least equal to 5 (The size of the instance), we can take 50 ants. Each ant must build a solution every iteration; indeed, each ant is tasked with examining and selecting objects and updating the trails.
4. The execution is given in the C++ programs.

8- Computational results after 50 excutions

Metaheuristic	Best found solution	Exact solution	Average runtime	$e_{rel}\%$
SA	25	25	0.094 s	0%
TS	25	25	0.2605 s	0%
ACO	25	25	0.1311 s	0%

Study the results: We observe that all metaheuristics perform well in terms of time and solution quality.

Exercise 3: Solving Assignment Problem with GA and VNS

(1)- The assignment problem: n persons (i) and n tasks (j), it costs C_{ij} to let person i do task j , the objective is to find the minimal cost assignment.

The model:

$$\max \sum_{i=1}^n \sum_{j=1}^n C_{ij} x_{ij}$$

$$\sum_{j=1}^n x_{ij} = 1 \quad , i = 1, \dots, n$$

$$\sum_{i=1}^n x_{ij} = 1 \quad , j = 1, \dots, n$$

(2)- The number of all possible solutions: $5! = 120$

(3)- Give a solution using the Greedy algorithm:

$$C = \begin{pmatrix} 20 & 15 & 18 & 20 & 25 \\ 18 & 20 & 12 & 14 & 15 \\ 21 & 23 & 25 & 27 & 25 \\ 17 & 18 & 21 & 23 & 20 \\ 18 & 18 & 16 & 19 & 20 \end{pmatrix}$$

C=12
2 do Task 3

$$C = \begin{pmatrix} 20 & 15 & \text{---} & 20 & 25 \\ \text{---} & \text{---} & \text{---} & \text{---} & \text{---} \\ 21 & 23 & \text{---} & 27 & 25 \\ 17 & 18 & \text{---} & 23 & 20 \\ 18 & 18 & \text{---} & 19 & 20 \end{pmatrix}$$

C=12+15=27
2 do Task 3, 1 do Task 2

$$C = \begin{pmatrix} \text{[Green]} & \text{[Green]} & \text{[Green]} \\ 21 & & 27 & 25 \\ \boxed{17} & & \text{[Green]} & \text{[Green]} \\ 18 & & 19 & 20 \end{pmatrix}$$

$C=12+15+17=44$
 2 do Task 3, 1 do Task 2,
 4 do Task 1

$$C = \begin{pmatrix} \text{[Green]} & \text{[Green]} & \text{[Green]} & \text{[Green]} \\ \text{[Green]} & \text{[Green]} & 27 & 25 \\ \text{[Green]} & \text{[Green]} & \text{[Green]} & \text{[Green]} \\ \text{[Green]} & \text{[Green]} & \boxed{19} & 20 \end{pmatrix}$$

$C=12+15+17+19+25=88$
 2 do Task 3, 1 do Task 2,
 4 do Task 1, 5 do Task 4
 and 3 do Task 5.

With: $C = 88$.

(4)- Propose solution encoding for the AP: Task assignment representation using a vector of length n (here it's 5) that encodes the tasks in the order of persons exciting them. For example, if we want to code the solution found by the greedy algorithm, it can be represented by: (2 — 3 — 5 — 1 — 4) it means: 1 do task 2, 2 do task 3, 3 do task 5, 4 do task 1 and 5 do task 4.

(5)- Genetic Algorithm

1. The mutation can be for example swapping two tasks. The crossover is the selection of two solutions randomly in the current generation.

2. The execution is given in the C++ programs.

(6)- Variable Neighborhood Search

1. Give $k_{max} = 3$ neighborhood structures to the shaking process
 - 1.1 **N1** Swapping move: Generate two random positions i and j , and then swap their elements in the sequence.
 - 1.2 **N2** Shifting move: Generate a random number d , and then move each task by d positions.
 - 1.3 **N3** Insertion move: Generate a random position i , and a random integer d , then move the element at position i to position d and update the positions of the other tasks.
2. For local search we can apply neighborhood structure **N3**.
3. The execution is given in the C++ programs.

7- Computational results after 50 excutions

Metaheuristic	Best found solution	Best known solution	Average runtime	$e_{rel}\%$
GA	87	87	0.2545 s	0%
VNS	90	88	0.0363 s	2.27%